

浙江大学实验报告

姓名：_____ 专业： 信息工程 _____ 学号： _____
课程名称： 数字系统设计实验 _____ 任课老师： 屈民军、唐奕 _____
实验名称： 音频播放器设计 _____

1 实验目的和要求

1.1. 实验目的

- (1) 掌握基于 FPGA 的复杂数字系统自顶向下的层次化设计方法。
- (2) 深入理解有限状态机 (FSM)、直接数字频率合成器 (DDS)、ROM 存储器定制以及分频电路的工作原理与应用。
- (3) 掌握音频编解码芯片 (Audio Codec) 接口与 I2C 通信协议的黑盒 (IP/网表) 调用方法。
- (4) 熟练使用 Vivado 软件进行工程管理、代码编写、逻辑综合、引脚约束及比特流下载测试。
- (5) 培养在实际硬件调试中发现问题、分析问题 (如时钟域交叉、底层硬件延迟、数据对齐失真等) 并解决问题的工程实践能力。

1.2. 实验要求

- (1) 分模块设计一个能够播放 4 首不同乐曲的音乐播放器。
- (2) 设置 play/pause_button、next_button、reset 三个按键，按下 play/pause_button 键，音乐在播放和暂停之间切换；按 next_button 播放下一首乐曲；且按键需经过防抖处理。
- (3) 利用 DDS 产生平滑的正弦波音频信号，并通过 I2S/Audio Interface 模块输出给音频 DAC 芯片进行播放。
对于该发生器，
采样频率 $f_c = 48\text{kHz}$ ；
正弦信号频率范围为 $20\text{Hz} \sim 20\text{kHz}$ ；
正弦信号序列宽度 16 位，包括一位符号；
- (4) 包含状态指示：LED 指示当前播放状态及歌曲编号。
- (5) 完成所有底层模块的 RTL 代码编写与仿真，最终在 FPGA 开发板上完成硬件验证。

2 ModelSim 工程设计

本次实验采用自顶向下的设计思路，将整个音乐播放器划分为顶层接口、次顶层播放逻辑、以及底层核心器件三个层级。

2.1. 总体设计架构

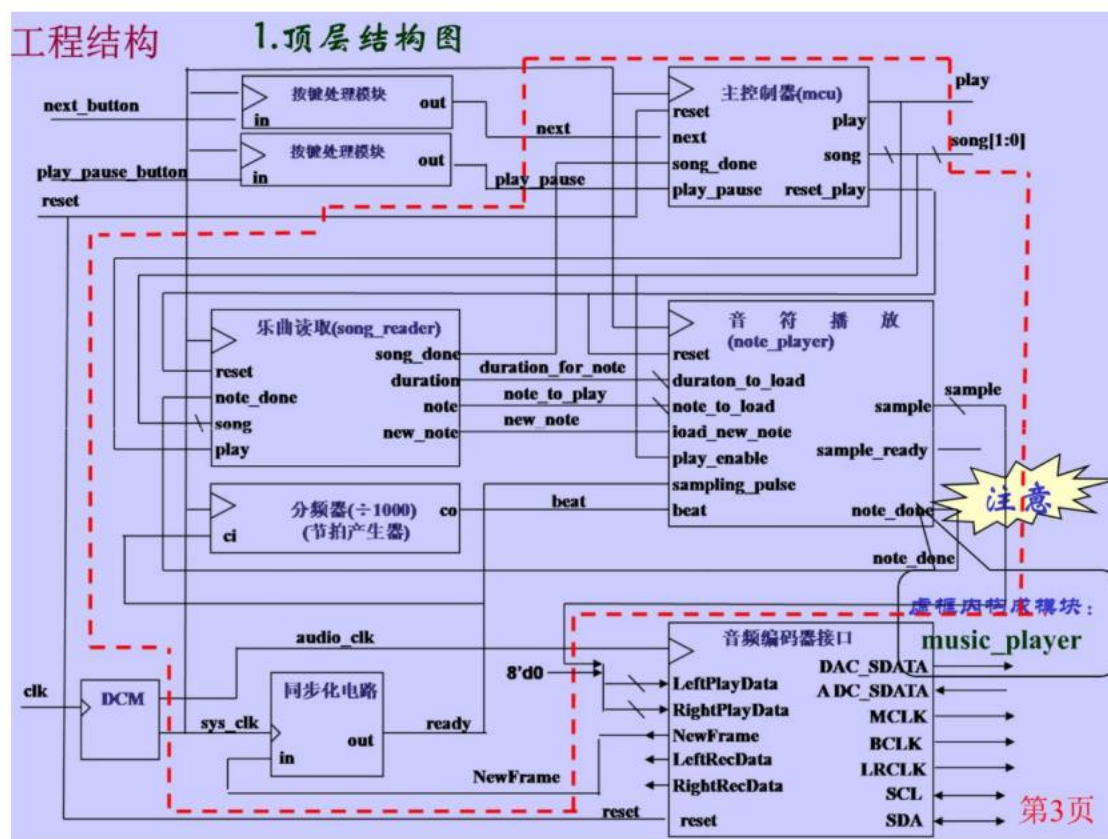
系统主要由以下模块构成：

- (1) MCU (主控状态机)：负责接收按键信号，控制播放器的状态 (复位、暂停、播放、下一首)，并输出相应的控制信号 (play, reset_play, next_song) 以及当前歌曲编号。
- (2) Song Reader (乐曲读取)：根据 MCU 给出的歌曲编号，利用计数器产生地址，从存

储乐谱的 ROM 中依次读出音符的音高（note）和持续时间（duration）。

- (3) Note Player（音符播放）：包含 DDS 核心模块。根据传入的音符音高频率控制字，在指定的持续时间内产生对应频率的正弦波采样点（sample）。
- (4) Audio Interface（音频接口）：提供的黑盒模块（.edf 网表），负责配置板载 Audio Codec 芯片，并将生成的数字正弦波转换为模拟音频信号驱动耳机。
- (5) 其他辅助模块：分频器、按键消抖模块等。

主架构如下：



2.2. 实验原理简述

各主要子模块作用如下：

- (1) 时钟管理模块(DCM)产生 100MHz 的系统时钟 sys_clk 和 12.5MHz 的音频时钟 audio_clk；
- (2) 主控制器(mcu)模块接收按键信息，控制 song_reader 模块是否播放及播放哪首乐曲；
- (3) 乐曲读取模块(song_reader)根据 mcu 模块的要求，逐个取出音符信息 {note,duration} 输入 note_player 模块播放，当一首乐曲播放完毕，传回 mcu 模块乐曲播放结束信号 song_done；
- (4) 音符播放模块(note_player)接收到需播放的音符，在音符的持续时间内，以 48kHz 送出该音符的正弦波样品至音频编解码接口模块。当一个音符播放结束，向 song_reader 模块发送一个 note_done 脉冲索取新的音符。
- (5) 音频编解码接口模块负责将音符的正弦波样品转换为串行输出并发送给音频编解码芯片 ADAU1761。音频编解码芯片 ADAU1761 接收正弦波样品，再进行 A/D 转换并放大，最后送至扬声器播放。note_player 模块产生的正弦波样品为 16 位二进制，需在低位加 8 个 0 后送入音频编解码接口模块。

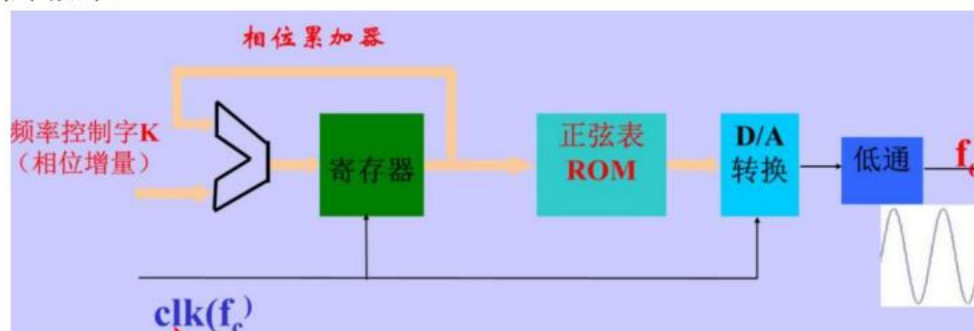
- (6) 节拍基准产生器产生 48Hz 的节拍定时基准脉冲信号(beat), 而 ready 信号频率为 48kHz, 因此, 节拍基准产生器即为分频比为 1000 的分频器。
- (7) 按键处理模块完成输入同步化、防颤动和脉宽变换等功能。

2.2.1. DDS 模块

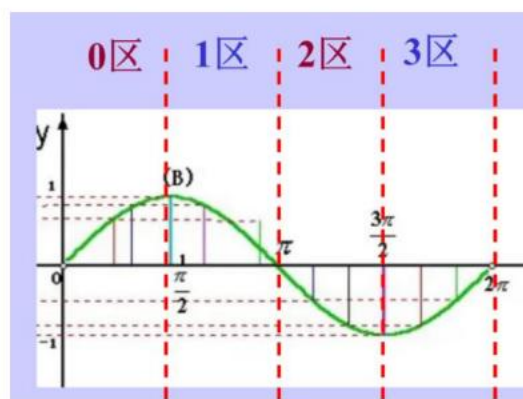
2.2.1.1. 基本原理

DDS 基于相位累加器原理。利用 48kHz 的采样索取脉冲 (NewFrame) 作为时钟, 将频率控制字不断累加产生相位值, 再以该相位值为地址去查找正弦波 ROM 表 (Sine ROM)。在此过程中, DDS 技术通过改变寻址的步长 (对数字波形查表的相位增量) 来改变输出信号的频率, 从而输出特定频率的正弦波数据。

原理框图如下:



输出正弦信号频率满足: $f_0 = \frac{K}{2^m}$, 其中 m 为 Sine ROM 地址线位数。要得到某一频率的正弦信号, 相位增量 K 值不一定为整数。因此, 为了得到更准确的正弦信号频率, K 的位数会增加几位小数。相位累加器的位数由 m 位整数和 p 位小数组成, 高 m 位整数部分作为 Sine ROM 的地址。根据 DDS 输出信号的最低频率要求, $m = 12$; 同时相位累加器位数会增加 10 位小数。所以, 相位累加器为 22 位累加器, 高 12 位为 Sine ROM 的地址。存储一个完整周期的正弦信号样品可采用 $2^{12} \times 16 \text{ bits}$ 的 ROM。但由于正弦波形的对称性, 可将正弦波形分为四个区域, 只需要在 Sine ROM 中存储四分之一的正弦信号 (0 区) 样品即可, 如下图所示:



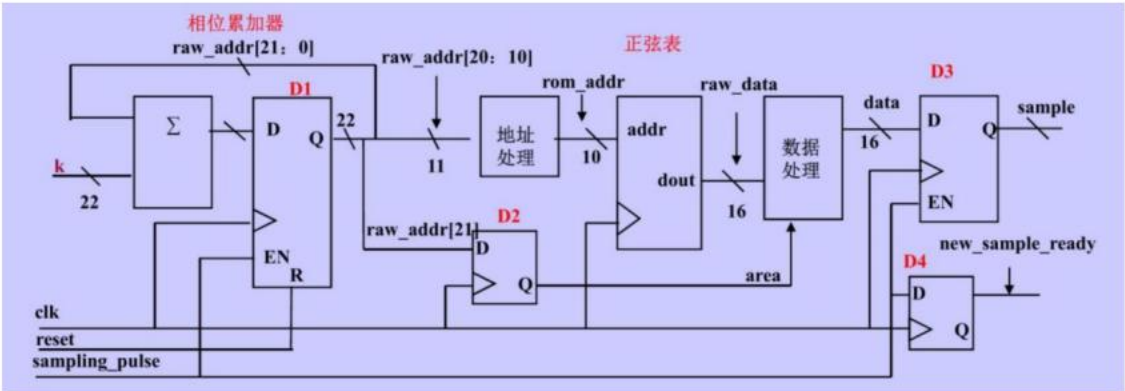
1 区: 与 0 区对称于 $\pi/2$ (第 1024 个样品), 即 1 区第 N 个样品与 0 区的第 (1024-N) 样品相同。

2 区: 2 区的第 N 个样品与 0 区的第 N 个样品样品幅度相同, 极性相反。

3 区: 3 区的第 N 个样品与 0 区的第 (1024-N) 样品幅度相反, 极性相反。

四分之一周期的正弦信号样品未给出 90° 的样品值, 因此在 ROM 地址为 1024 时可取地址为 1023 的值 (实际上地址为 1023 时, 正弦信号样品已达最大值)。

2.2.1.2. 总体设计



相位累加得到 22 位原始地址，整数部分即为完整周期正弦信号样品的地址，其中高两位地址可区分正弦的四个区域。由于 sine_rom 只保存了四分之一周期的 1024 个样品，所以整数部分不能直接作为 sine_rom 地址，必须处理。

正弦区域	sineROM 地址	正弦样品 data	
0	raw_addr [19:10]	raw_data [15:0]	
1	当 raw_addr [20:10] = 1024 时，rom_addr 取 1023，其他情况取~raw_addr [19:10]+1	raw_data [15:0]	地址镜像翻转
2	raw_addr [19:10]	~raw_data [15:0] + 1	数据取反
3	当 raw_addr [20:10] = 1024 时，rom_addr 取 1023，其他情况取~raw_addr [19:10] + 1	~raw_data [15:0] + 1	地址镜像翻转且数据取反

2.2.1.3. Verilog 代码

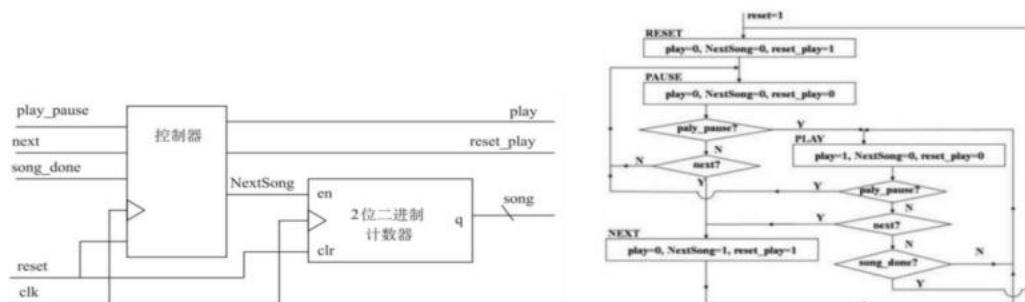
```

1  `timescale 1ns/1ps
2  // DDS 正弦信号发生器, 使用 1/4 周期 ROM 通过象限对称生成完整周期
3  module dds (
4      input          clk,
5      input          reset,
6      input          sampling_pulse,
7      input [21:0] k,
8      output [15:0] sample,
9      output          new_sample_ready
10 );
11
12 localparam PHASE_W = 22; // 相位累加器位宽
13 localparam ADDR_W = 10; // ROM 地址位宽
14 localparam PHASE_HIGH_W = ADDR_W + 2; // 相位高12位
15
16 wire [PHASE_W-1:0] phase_acc, phase_next;
17 wire [PHASE_HIGH_W-1:0] phase_high = phase_acc[PHASE_W-1:PHASE_W-PHASE_HIGH_W]; // 取高12位
18 wire [ADDR_W-1:0] rom_addr;
19 wire sign;
20 wire [15:0] rom_data; // ROM 原始数据
21 wire [15:0] signed_data; // 符号处理后的数据
22
23 // 1. 相位累加器加法器 (22位加法)
24 full_adder_n #(WIDTH(PHASE_W)) adder (
25     .a(phase_acc), .b(k), .ci(0), .s(phase_next), .co()
26 );
27
28 // 2. 相位累加器寄存器
29 dffre #(n(PHASE_W)) ph_reg (
30     .clk(clk), .r(reset), .en(sampling_pulse), .d(phase_next), .q(phase_acc)
31 );
32
33 // 3. 地址处理器
34 addr_process #(ADDR_WIDTH(ADDR_W)) addr_proc (
35     .phase_high(phase_high), .rom_addr(rom_addr), .sign(sign)
36 );
37
38 // 4. 正弦查找表 ROM
39 sine_rom rom (
40     .clk(clk), .addr(rom_addr), .dout(rom_data)
41 );
42
43 // 5. 符号处理: 负半周取补码
44 assign signed_data = sign ? (~rom_data + 1'b1) : rom_data;
45
46 // 6. 输出样品寄存器
47 dffre #(n(16)) samp_reg (
48     .clk(clk), .r(reset), .en(sampling_pulse), .d(signed_data), .q(sample)
49 );
50
51 // 7. 新样品就绪信号
52 dffre #(n(1)) ready_reg (
53     .clk(clk), .r(reset), .en(1'b1), .d(sampling_pulse), .q(new_sample_ready)
54 );
55 endmodule

```

2.2.2. MCU 模块

状态机包含四个状态: RESET、PAUSE、PLAY、NEXT。在 PAUSE 时按下 play_pause 键进入 PLAY; 在 PLAY 时按 next 键进入 NEXT, 随后无条件回到 PLAY 自动播放下一首, 在 PLAY 时按下 play_pause 可以回到 PAUSE 状态。



结合 ASM 图, 得到 Verilog 代码如下:
(代码删去了部分换行, 以便截图显示)

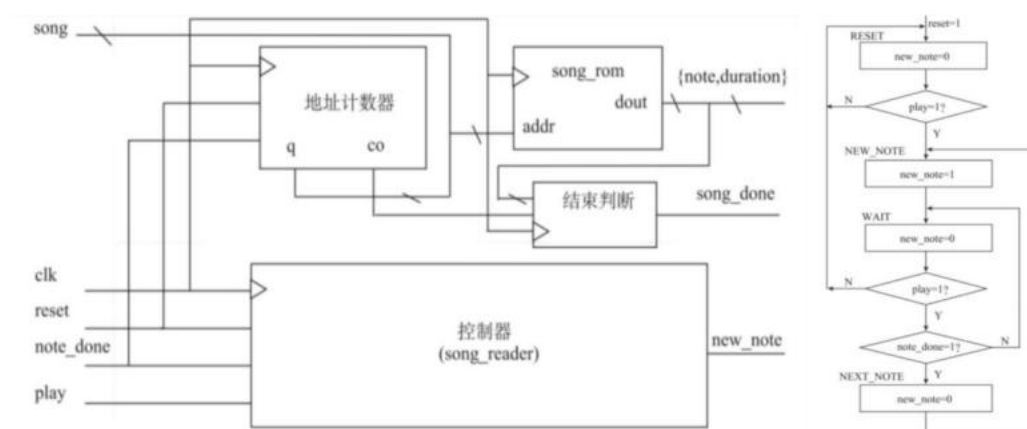
```

1 `timescale 1ns / 1ps
2 // mcu (主控单元顶层)
3 module mcu(input wire clk,input wire reset,input wire play_pause,input wire next,input wire song_done,output wire play_pause,
4 output wire next_song);
5 // 实例化控制器
6 controller u_controller(.clk(clk),.reset(reset),.play_pause(play_pause),.next(next),.song_done(song_done),.play(play),
7 // 实例化歌曲计数器
8 song_counter u_song_counter(.clk(clk),.reset(reset),.next_song(next_song),.song(song));
9 endmodule
10 // 子模块: controller (有限状态机)
11 module controller(input wire clk,input wire reset,input wire play_pause,input wire next,input wire song_done,output wire play_pause,
12 output reg reset_play,
13 output reg next_song);
14 );
15 // 4个状态
16 parameter RESET = 2'b00, PAUSE = 2'b01, PLAY = 2'b10, NEXT = 2'b11;
17 reg [1:0] state, next_state;
18 // 1. 状态寄存器
19 always @(posedge clk) begin
20 if (reset)
21 state <= RESET;
22 else
23 state <= next_state;
24 end
25 // 2. 次态逻辑
26 always @(*) begin
27 case (state)
28 RESET: next_state = PAUSE;
29 PAUSE: begin
30 if (play_pause) next_state = PLAY;
31 else if (next) next_state = NEXT;
32 else next_state = PAUSE;
33 end
34 PLAY: begin
35 if (play_pause) next_state = PAUSE;
36 else if (next) next_state = NEXT;
37 else if (song_done) next_state = RESET;
38 else next_state = PLAY;
39 end
40 // 切歌(NEXT)后, 无条件进入播放(PLAY)
41 NEXT: next_state = PLAY;
42 default: next_state = RESET;
43 endcase
44 end
45 // 3. 输出逻辑
46 always @(*) begin
47 case (state)
48 RESET: begin play = 0; reset_play = 1; next_song = 0; end
49 PAUSE: begin play = 0; reset_play = 0; next_song = 0; end
50 PLAY: begin play = 1; reset_play = 0; next_song = 0; end
51 NEXT: begin play = 0; reset_play = 1; next_song = 1; end
52 default: begin play = 0; reset_play = 0; next_song = 0; end
53 endcase
54 end
55 endmodule
56 // 子模块: song_counter (2位歌曲计数器)
57 module song_counter(
58 input wire clk,
59 input wire reset,
60 input wire next_song,
61 output reg [1:0] song);
62 );
63 always @(posedge clk) begin
64 if (reset)
65 song <= 2'b00;
66 else if (next_song)
67 song <= song + 1'b1;
68 end
69 endmodule

```

2.2.3. song_reader 模块

song_reader 模块结构框图与控制器算法流程图如下所示:



`song_rom` 是一个只读存储器, 用来存放乐曲, 容量为 $27 \times 12\text{bits}$, 共存放四首乐曲, 每首乐曲占用 $27 \times 12\text{bits}$ 空间, 即每首乐曲最长由 32 个音符组成。因此, `song_rom` 高 2 位地址决定哪首乐曲, 而低 5 位地址决定这首乐曲的哪个音符。`song_rom` 每个地址存放一个

音符信息，音符信息由 12 位二进制 组成，高 6 位表示音符标记 `note`，低 6 位表示音长 `duration`。

地址计数器为 5 位二进制计数器，其中 `note_down` 为计数器使能输入，当 `note_down` 为高电平时允许计数。

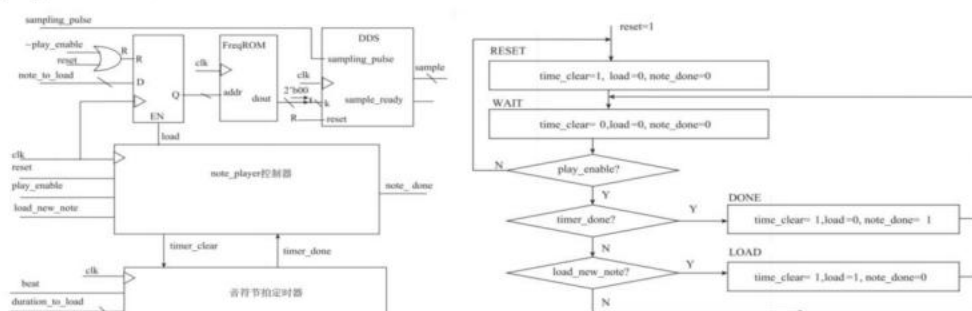
当地址计数器出现进位或 `duration` 为 0 时，表示乐曲结束，应输出一个时钟周期宽度的高电平脉冲信号；结束判断模块采用状态机实现脉宽变换电路。

2.2.4. note_player 模块

音乐播放模块 `music_player` 是本实验的核心模块，它主要任务有：

- (1) 从 `song_reader` 模块接收需播放的音符 {`note`, `duration`}；
- (2) 根据 `note` 值找出 DDS 的相位增量 `k`；
- (3) 以 48kHz 从 Sine ROM 取出正弦样品送给音频编解码器接口模块；
- (4) 当一个音符播放完毕，向 `song_reader` 模块索取新的音符。

`note_player` 结构框图如图所示：



`FreqROM` 为只读存储器，完成音符标记 `note` 与 DDS 模块的相位增量 `k` 查找表关系。

`note_player` 控制器负责与 `song_reader` 模块接口，读取音符信息，并根据音符信息从 `Frequency ROM` 中读取相应相位增量 `k` 送给 DDS 子模块。另外，`note_player` 控制器还需要控制音符播放时间。`note_player` 控制器的算法流程如上右图所示：

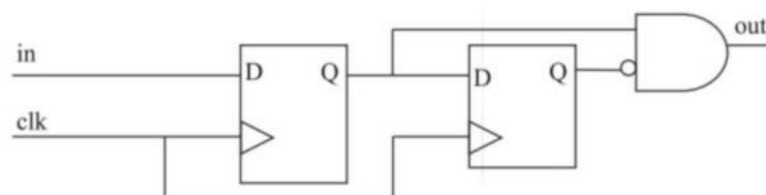
音符定时器为 6 位二进制计数器，`beat`、`timer_clear` 分别为使能、清 0 信号，均为高电平有效；定时时间由音长信号 `duration_to_load` 决定，即 `duration_to_load-1` 个 `beat` 周期，`timer_done` 为定时结束标志。

子模块 DDS 的功能就是利用 DDS 技术产生正弦样品，需要注意的是，DDS 模块的输入 `k` 为 22 进制，因此需要 `FreqROM` 输出的 20 位相位增量高位加 2 个 0 后接入 DDS。

2.2.5. 同步化电路

音频编解码接口模块的输出信号 `NewFrame` 的脉冲宽度为一个 `audio_clk` 时钟周期，需通过同步化处理，

产生与 `sys_clk` 同步且脉冲宽度为一个 `sys_clk` 时钟周期的信号 `ready`。电路由同步器和脉冲宽度变换电路组成：



2.2.6. 时钟管理模块

IP 内核时钟管理模块的输入时钟 `clk` 频率为 100MHz，产生 100MHz 的系统时钟和 12.5MHz

的音频时钟。

2.2.7. 节拍基准产生器模块

节拍基准产生器产生 48Hz 的节拍定时基准脉冲信号(beat)，而 ready 信号频率为 48kHz，因此节拍基准产生器即为一个分频比为 1000 的分频器。

2.2.8. 次顶层模块

用于统一各子模块的功能，并进行 Modelsim 仿真。

```
1 `timescale 1ns / 1ps
2 //将 mcu, divider, 乐谱读取和音符播放器连接在一起
3 module music_player #(parameter sim = 0) (input wire clk,input wire reset,input wire play_pause,input wire next,input
4 // 内部连线
5 wire beat;
6 wire sampling_pulse;
7 wire play_enable;
8 wire reset_play;
9 wire note_done;
10 wire new_note;
11 wire [5:0] note;
12 wire [5:0] duration;
13 wire song_done;
14 // 1. 同步采样脉冲
15 syn u_syn (
16     .clk (clk),
17     .in (NewFrame),
18     .out (sampling_pulse)
19 );
20 // 2. 节拍发生器
21 localparam DIV_N = (sim == 1) ? 64 : 2000000;
22 localparam DIV_BITS = (sim == 1) ? 6 : 21;
23 divider #(
24     .n(DIV_N),
25     .counter_bits(DIV_BITS)
26 ) u_divider (
27     .clk (clk),
28     .reset (reset),
29     .clk_out (beat)
30 );
31 // 3. 主控状态机 mcu
32 mcu u_mcu (
33     .clk (clk),
34     .reset (reset),
35     .play_pause (play_pause),
36     .next (next),
37     .song_done (song_done),
38     .play (play_enable),
39     .song (song),
40     .reset_play (reset_play)
41 );
42 assign play = play_enable;
43 // 4. 乐谱读取器
44 song_reader u_song_reader (
45     .clk (clk),
46     .reset (reset_play),
47     .play (play_enable),
48     .song (song),
49     .note_done (note_done),
50     .song_done (song_done),
51     .note (note),
52     .duration (duration),
53     .new_note (new_note)
54 );
55 // 5. 音符播放器
56 note_player u_note_player (
57     .clk (clk),
58     .reset (reset_play),
59     .play_enable (play_enable),
60     .note_to_load (note),
61     .duration_to_load (duration),
62     .load_new_note (new_note),
63     .beat (beat),
64     .sampling_pulse (sampling_pulse),
65     .note_done (note_done),
66     .sample (sample),
67     .sample_ready ()
68 );
69 endmodule
```

2.2.9. 顶层模块

用于统一顶层与 DCM 时钟模块，进行 Vivado 综合。


```

1 `timescale 1ns / 1ps
2 //top_music_player
3 module top_music_player (input wire clk,input wire reset_btn,
4 //未消抖的原始输入
5 input wire btn_play_in,input wire btn_next_in,
6 // 音频编解码器
7 input wire ADC_SDATA,output wire DAC_SDATA,output wire BCLK,output wire MCLK,output wire LRCLK,inout wire SCL,inout
8 // LED 状态指示
9 output wire led_play,output wire [1:0] led_song);
10 wire sys_clk; // 系统逻辑时钟
11 wire audio_clk; // 由 DCM 生成的音频时钟
12 wire locked; // DCM 锁定信号
13 wire sys_reset; // 复位
14 wire btn_play_pulse;wire btn_next_pulse;wire NewFrame;wire [15:0] sample;
15 // 1. 实例化DCM
16 sys_dcm u_dcm (
17 .clk_in1 (clk),
18 .reset (reset_btn),
19 .clk_out1 (sys_clk),
20 .clk_out2 (audio_clk),
21 .locked (locked)
22 );
23 // 只有锁相环稳定了才撤销复位
24 assign sys_reset = reset_btn | (~locked);
25 // 2. 实例化按键处理模块 (接 sys_clk)
26 button_process_unit u_btn_play (
27 .clk (sys_clk), // 使用 sys_clk
28 .reset (sys_reset),
29 .ButtonIn (btn_play_in),
30 .ButtonOut (btn_play_pulse)
31 );
32 button_process_unit u_btn_next (
33 .clk (sys_clk),
34 .reset (sys_reset),
35 .ButtonIn (btn_next_in),
36 .ButtonOut (btn_next_pulse)
37 );
38 // 3. 实例化音频编解码器接口 (接 audio_clk)
39 // 低位填充
40 wire [23:0] play_data = {sample, 8'd0};
41 AudioInterface u_audio_interface (
42 .clk (audio_clk), //接 audio_clk
43 .reset (sys_reset),
44 .ADC_SDATA (ADC_SDATA),
45 .DAC_SDATA (DAC_SDATA),
46 .BCLK (BCLK),
47 .MCLK (MCLK),
48 .LRCLK (LRCLK),
49 .SCL (SCL),
50 .SDA (SDA),
51 .error (),
52 .LeftPlayData (play_data),
53 .RightPlayData (play_data),
54 .LeftRecData (),
55 .RightRecData (),
56 .NewFrame (NewFrame)
57 );
58 // 4. 实例化次顶层
59 music_player #(1) u_music_player (
60 .clk (sys_clk), // 使用 sys_clk
61 .reset (sys_reset),
62 .play_pause (btn_play_pulse),
63 .next (btn_next_pulse),
64 .NewFrame (NewFrame),
65 .sample (sample),
66 .play (led_play),
67 .song (led_song)
68 );
69 endmodule

```

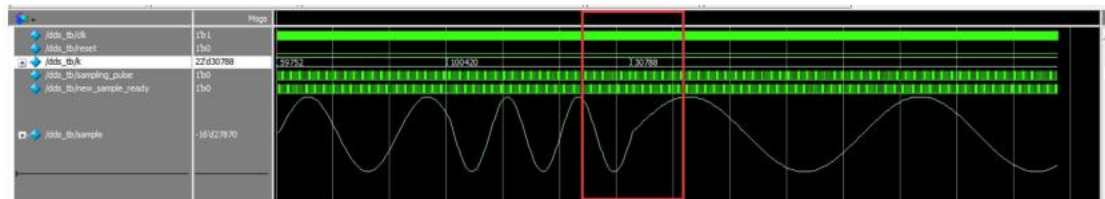
3 主要仪器设备

- (1) 安装了 Modelsim 2020.4 和 Vivado 2020.1 的计算机;
- (2) Nexys Video Artix-7 FPGA 开发板;
- (3) 耳机

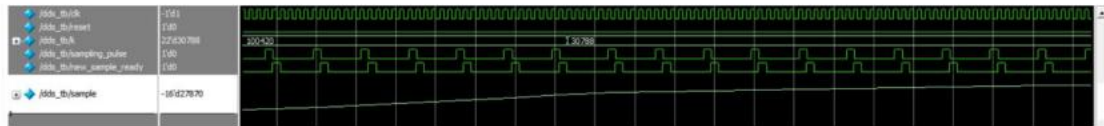
4 仿真波形分析

4.1. DDS 模块

建立 Modelsim 工程并执行仿真后，得到如下波形：



节选下面波形分析：



(1) 采样脉冲:

观察微观波形, `sampling_pulse` 表现为周期性到来的单周期高电平脉冲(代表外部 48kHz 的采样数据索取请求)。

当 `sampling_pulse` 到来时, DDS 内部的相位累加器执行一次加 `k` 运算, 并触发后续的 ROM 地址映射与查表操作。

在 `sampling_pulse` 发生后的下一个时钟周期, 输出信号 `new_sample_ready` 稳定拉高一个周期。这说明系统遵循了 D 触发器的延迟特性, 在正弦波样本 (`sample`) 查表完成并锁存后, 才向外部发出“数据就绪”的应答脉冲。

(2) 频率转折点的“相位连续性”:

在微观波形中 `k` 值突变的交界处(十进制值由 100420 突变为 30788 的瞬间):

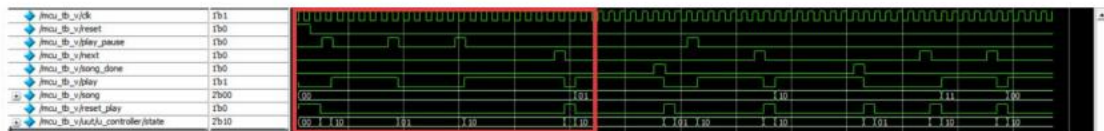
在频率控制字 `k` 发生改变的瞬间, 正在上升的正弦波 `sample` 并没有发生断裂, 也没有因为频率切换而突变归零。

这正是 DDS 技术相较于传统频率合成器的最大优势: 相位连续。因为 `k` 的改变仅仅是改变了下一次累加的“步长”, 而存储当前波形位置的“相位累加器”内部的绝对数值并没有被清零。因此, 波形会紧接着前一个时钟周期的电压幅值, 以一个全新的平缓斜率继续平滑地上升。

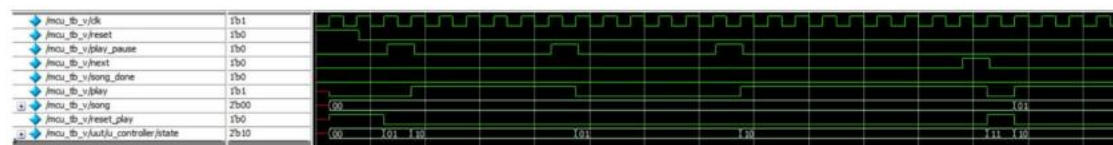
这种相位连续的频率切换, 从频谱角度看不会产生任何高频毛刺, 在实际声学表现中保证了相邻音符过渡时的丝滑, 消除了“破音”现象。

4.2. MCU 模块

仿真波形如下:



节选以下部分进行分析:



(1) 复位与初始状态:

观察波形最左侧, 当全局异步复位信号 `reset` 为高电平(1)时, 状态机 `state` 保持为 00(代表 RESET 状态)。此时输出信号 `reset_play` 为高电平, 正确执行了对底层模块的复位操作。当 `reset` 拉低后的下一个时钟上升沿, 状态机自动跳转至 01(代表 PAUSE 状态), `reset_play` 恢复为 0, 符合预期。

(2) 播放/暂停逻辑:

当系统处于 01(PAUSE) 状态时, 给出一个 `play_pause` 高电平脉冲, 在下一个时钟上升沿, `state` 成功跳转至 10(代表 PLAY 状态), 同时观察到输出信号 `play` 拉高为 1, 音乐开始播放。

随后再次给出一个 `play_pause` 脉冲, `state` 从 10 准确跳回 01(PAUSE), 同时 `play` 信号拉低为 0。该部分的双向切换逻辑正确。

(3) 切歌逻辑(NEXT)与自动播放:

在波形中段，当系统处于 01 (PAUSE) 状态时，给出一个 next 高脉冲信号。

在下一个时钟沿，`state` 成功进入 11（代表 NEXT 状态）。此时观察到 `reset_play` 产生了一个周期的脉冲以复位底层播放器，且歌曲计数器 `song` 由 00 变为了 01，说明切歌计数功能正常生效。

观察离开 11 (NEXT) 状态的下一个时钟沿, 发现 `state` 没有停留在暂停, 而是直接跳转到了 10 (PLAY) 状态, `play` 信号随之拉高。这完成了状态转移图中“切歌后自动播放”的转移逻辑。

(4) 歌曲结束 (song_done) 逻辑:

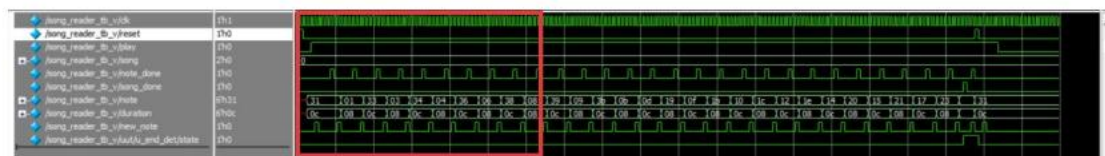
在波形最右侧，模拟底层乐谱读取完毕，给出一个 `song_done` 脉冲信号。

此时系统正处于 10 (PLAY) 状态, 接收到信号后, 在下一个时钟沿准确跳转回了 00 (RESET) 状态, reset play 再次拉高。

随后系统按流程自动进入 01 (PAUSE) 状态等待下一次指令。

4.3. song_reader 模块

仿真波形如下:



节选下面部分进行分析:



(1) 复位与启动读取:

观察波形最左侧,当全局异步复位信号 `reset` 拉低撤销复位,且接收到主控模块传来的 `play = 1` (允许播放)指令后,内部状态机 `state` 从 0 (代表 RESET 状态)跳转至 1 (代表 NEW NOTE 状态)。

在状态 1 下，系统成功输出一个时钟周期的 `new_note=1` 高脉冲（通知后续模块装载新音符）。同时，地址计数器选中 ROM 的初始地址，正确输出了第一组音符数据（例如图中显示的 `note=6'b110001`，`duration=6'b001100`）。随后状态机自动进入 2（代表 WAIT 状态）待机。

(2) 循环读取逻辑:

当系统处于 2 (WAIT 待机) 状态期间, 接收到来自 `note_player` 模块发出的 `note_done=1` (当前音符播放完毕) 的高电平脉冲。

在下一个时钟上升沿，state 准确跳转至 3（代表 NEXT_NOTE 状态），此时内部地址计数器加 1。

紧接着的一个时钟沿, `state` 自动回到 1 (NEW_NOTE), 再次发出 `new_note=1` 脉冲, 并从 ROM 中顺利读出下一个音符的数据 (如波形中数据更新为了 `note=6'b000001`, `duration=6'b001000`)。

波形中连续多次展现了 WAIT(2) -> NEXT_NOTE(3) -> NEW_NOTE(1) -> WAIT(2) 的完整循环，证明了两个模块之间的请求与下发逻辑正确。

(3) 乐曲结束 (song done) 判定:

观察波形最右侧末端,当地址计数器不断递增产生溢出进位,或读取到标志结束的 duration = 6'b000000 时,内部的结束检测器(end detector)敏锐捕捉到了终止条件,准确输出了一

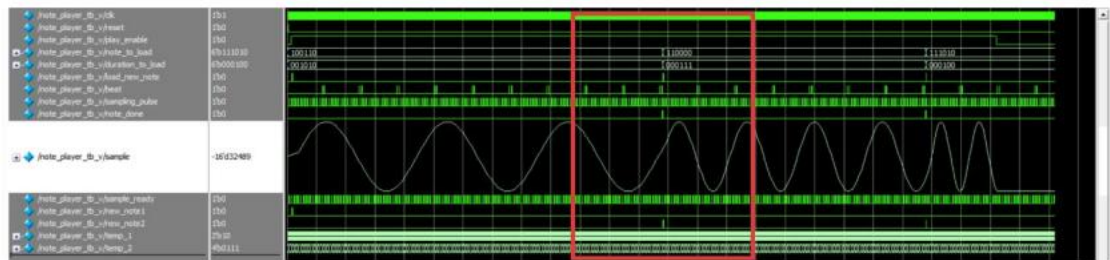
个时钟周期的 `song_done = 1` 高电平脉冲，用以通知 MCU 歌曲结束。
与此同时，状态机在 2 (WAIT) 状态下检测到结束信号后，跳回 0 (RESET) 状态，停止输出并等待主控模块的下一次播放指令。

结论：

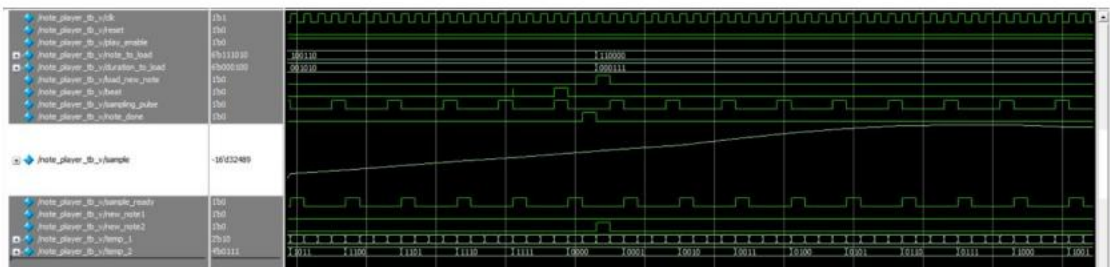
综上分析，`song_reader` 模块的 ROM 寻址输出稳定，提取的音调与音长数据清晰无误。状态机的流转 (0 → 1 → 2 → 3 → 1...) 与 ASM 图高度契合，实现了乐谱数据的自动化按序下发与检测。

4.4. `note_player` 模块

仿真波形如下：



节选以下部分重点分析：



(1) 音符加载与启动播放：

观察波形左侧，当 `play_enable`（允许播放）拉高后，伴随着 `load_new_note = 1` 脉冲的到来，系统成功将待播放的音符数据（例如 `note_to_load = 6'b100110`，对应十进制 38，低频音）和持续时间（`duration_to_load`）锁存入内部寄存器。

随后，底层的 DDS 模块解除复位，`sample` 端口开始持续输出平滑的正弦波形，启动逻辑正确。

(2) 频率控制与节拍倒计时：

频率响应：由波形可见，当加载的 `note_to_load` 值较小时（如 38），生成的正弦波周期极短、波形十分密集；而在波形后段加载了音高代码（如 `note_to_load = 6'b110000`，即 48）时，正弦波的周期变长。这验证了 DDS 通过改变相位增量来改变输出频率的原理。

时长控制：内部的节拍计数器根据 `duration_to_load` 进行倒计时。当计数达到指定节拍数后，系统准确输出了单周期的 `note_done = 1` 高电平脉冲，向系统索取下一个音符。

(3) 停播时的“波形归零”现象：

仔细观察波形的最右端，当 `play_enable` 信号拉低（代表暂停或播放结束）时，原本正在振荡的 `sample` 正弦波并没有走完完整的正弦周期，而是发生跳变。

这是 RTL 代码中设计的复位保护（`assign dds_reset = reset | (~play_enable);`）。一旦停止播放，DDS 内部的相位累加器被立刻清零，使得地址直接指向 `sin(0)=0`，切断输出。

(4) 音符切换时的频率跳变与相位连续性：

根据放大版波形图（频率跳变交界点）可以清晰地观察到两个连续音符交接瞬间的逻辑：波形中，前一个音符播放结束时，系统发出 `note_done = 1` 脉冲。紧接着，上游模块送来新

的音符数据（note_to_load 从 6'b100110 变为 6'b110000，duration_to_load 更新为 6'b000111），同时给出了 load_new_note = 1 的加载指令。

频率瞬间响应：在 load_new_note 脉冲之后，底层 DDS 模块的频率控制字（相位增量 k）更新。肉眼可见 sample 输出的正弦波形从较缓的低频波变陡为高频波，频率响应极具实时性。

观察正弦波在切换频率瞬间的交界点，波形并没有突变归零或者发生垂直跳变，而是顺着前一个音符结束时的相位点（电压幅值），以新的斜率（频率）平滑地继续振荡。

这展示了 DDS（直接数字频率合成）技术的理论优势——相位连续的频率切换。因为系统在切换相邻音符时并没有复位 DDS 内部的相位累加器，从而保证了音频在过渡时的绝对平滑，消除了音符衔接处的爆音。

4.5. 同步化电路

波形如下：



(1) 异步信号的跨域捕获与同步化：

观察波形中的 in 信号（模拟音频接口传来的 NewFrame），可以清晰地看到它的上升沿发生时刻与系统本地时钟 clk 的边缘并不对齐，呈现出异步特征。

当 in 信号拉高后，输出并没有立刻发生变化，而是等待到了下一个 clk 的上升沿。这是因为电路内部的第一级 D 触发器对异步信号执行了“打拍（Registering）”操作，将其同步到了本地的 sys_clk 时钟，隔离了跨时钟域传输时的亚稳态风险。

(2) 宽脉冲到单周期脉冲的“边沿检测”：

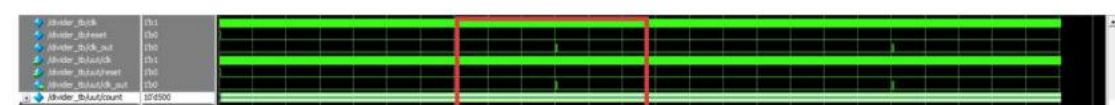
观察波形中段与后段：输入激励信号 in 保持了非常漫长的高电平状态。

尽管输入信号 in 的高电平宽窄不一，但输出信号 out 永远只在 in 拉高后的第一个有效时钟周期内，产生了一个宽度为 1 个 clk 周期的正脉冲，随后回落至 0 并保持，直至下一次上升沿的到来。

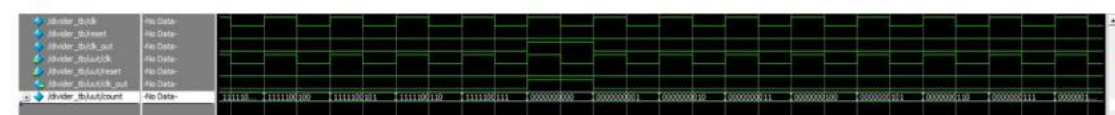
这种现象得益于 RTL 源码中设计的两级移位寄存器及 assign out = q1 & (~q2); 的组合门逻辑。当异步输入刚被捕获时，第一拍寄存器 q1 置 1，而第二拍寄存器 q2 仍保留着上一状态的 0（此时 ~q2 为 1），与门条件成立，out 吐出高电平；紧接着的下一个时钟沿，q2 也更新为 1，与门条件使 out 强制拉低。该结构构成了上升沿检测器。

4.6. 1000 分频器模块

波形如下：



部分波形放大如下：



(1) 计数器：

波形最下方展开的内部状态寄存器 /uut/count，在系统时钟 clk 的驱动下，该计数器在每个时钟周期的上升沿不断进行 +1 的线性累加操作。

波形清晰地记录了 count 从二进制 1111100100（十进制 996）一路递增至 1111100111（十进制 999）的完整过程。这里的数值 999 对应了 RTL 代码中设定的临界触发条件 (n - 1) 即 1000 - 1 = 999。

(2) 同步清零与单周期脉冲：

当 count 达到 999 后，在紧接着的下一个 clk 上升沿，计数器并没有继续累加到 1000，而是触发了同步清零逻辑，回归至 0000000000。

伴随计数器清零的同一个时钟上升沿，输出信号 clk_out 被精准拉高为 1。

clk_out 保持高电平刚好经过一个 clk 周期的时间。当下一个上升沿到来时，count 开始向 1 递增，clk_out 随之回落至 0。

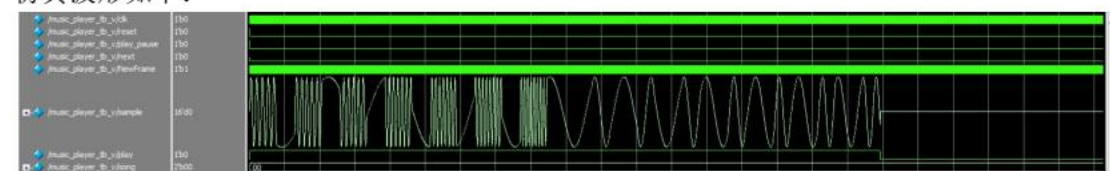
(3) 分频周期：

从波形的宏观逻辑可以推算出，系统从 count=0 计数到 count=999，刚好经历 1000 个时钟周期。

每 1000 个周期产生一次持续仅一拍的 clk_out 高电平，实现 $f_{out} = \frac{f_{in}}{1000}$ 的频率。

4.7. 次顶层模块

仿真波形如下：



(1) 发声：

系统在接收到启动指令后，MCU 的 play 信号拉高，系统正式进入 PLAY 状态。

随后，底层的 DDS 模块根据 song_reader 依次下发的音符频率字，在 sample 端口输出了一连串频率和包络各不相同的正弦波列。这代表各个不同音高的音符正在被正确播放。

(2) 播放结束时的自动切断：

观察波形骤停交界点，该时刻的输入激励：play_pause 与 next 信号均保持为低电平（0），外界并没有进行任何人为的按键中断操作。

尽管无外部中断，但输出控制信号 play 自动从 1 拉低到了 0。这是因为当前乐曲的最后一个音符播放完毕，底层 song_reader 发出了 song_done 结束脉冲。MCU 状态机在捕捉到该信号后，从 PLAY 态跳转至 RESET 并切入 PAUSE 状态。

在 play 信号拉低的瞬间，原本正在振荡的模拟正弦波 sample 被拉回 16'd0 的水平基准线。这验证了底层模块硬复位逻辑，避免了播放结束后的残留杂音。

(3) 仿真时间轴分布：

在全景图中可以观察到，波形前段极短的时间内完成了发声，而后续占据总仿真时间约 99% 的区域均为零电平直线。

这一现象与测试台（music_player_tb.v）的设定吻合，为了加快仿真速度，测试台中实例化时传入了 #(sim(1)) 参数。这使得底层节拍分频器的分频比从 2,000,000 减至 64，整首歌曲的播放时间被快进了。

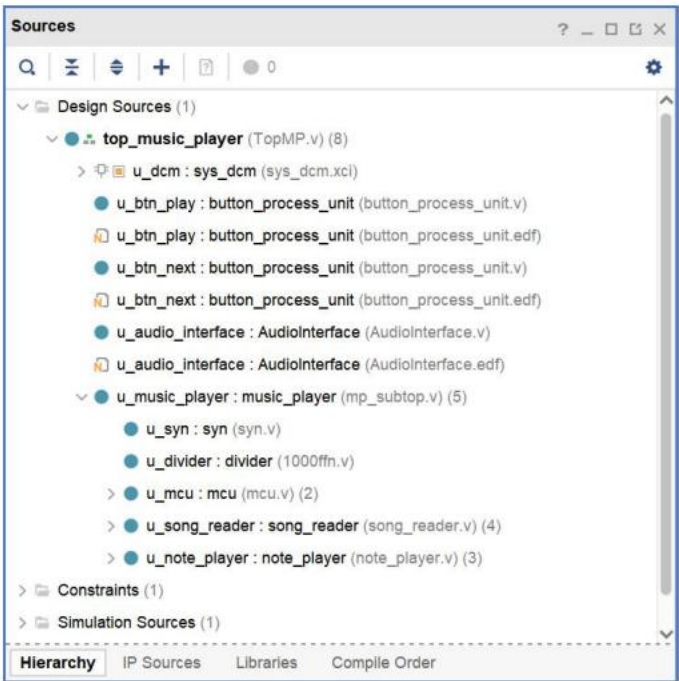
结果是，歌曲在仿真初期的极短时间内就播放完毕并触发了 song_done，系统切入 PAUSE 待机。然而，测试台的主循环（即代码中设定的 repeat (20000) 循环）仍在继续执行。

在 20000 次剩余循环时间里，尽管外部测试台仍在不断提供系统时钟 clk 和采样索取脉冲 NewFrame，但处于待机状态的片上系统锁住了输出，展现出了极好的静音特性，无漏电或寄生振荡。

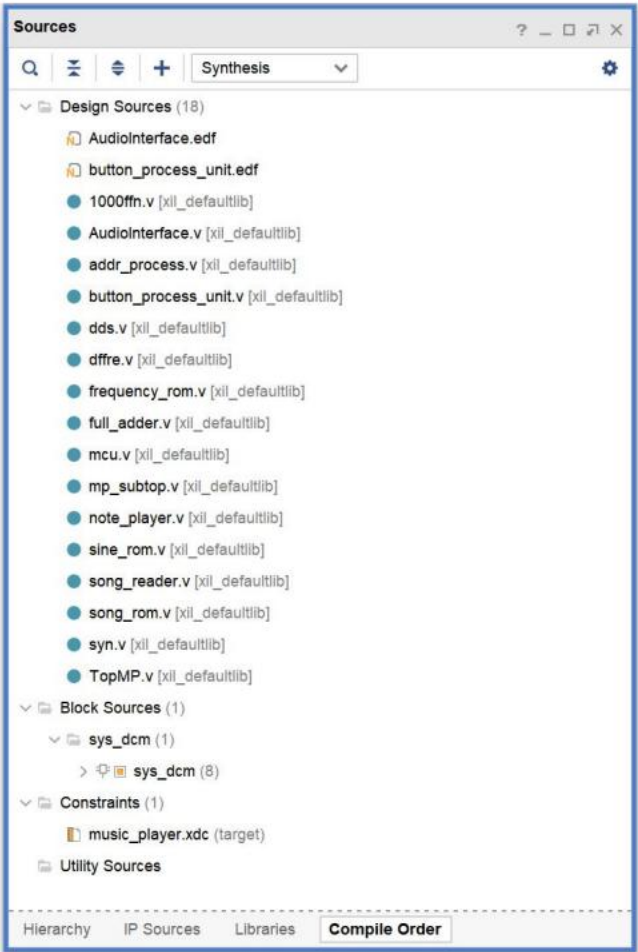
5 Vivado 工程实现与上板

建立工程文件之后，添加 ModelSim 文件和按键处理模块、音频编解码模块的网表文件与端

口声明并建立 IP 核，应显示如下的工程层次：



有如下的文件组成：



IP Core 的设置应该如下：

Re-customize IP

Clocking Wizard (6.0)

Documentation IP Location Switch to Defaults

IP Symbol Resource

☒ Show disabled ports

Component Name sys_dcm

Clocking Options Output Clocks Port Renaming MMCM Settings Summary

Clock Monitor

☐ Enable Clock Monitoring

Primitive

☒ MMCM ☐ PLL

Clocking Features

☒ Frequency Synthesis ☐ Minimize Power

☒ Phase Alignment ☐ Spread Spectrum

☐ Dynamic Reconfig ☐ Dynamic Phase Shift

☐ Safe Clock Startup

Jitter Optimization

☒ Balanced ☐ Minimize Output Jitter ☐ Maximize Input Jitter filtering

Dynamic Reconfig Interface Options

☐ Phase Duty Cycle Config ☐ Write DRP registers

☒ AXI4Lite ☐ DRP

Input Clock Information

Input Clock	Port Name	Input Frequency(MHz)	Jitter Options	Input Jitter	Source
☒ Primary	clk_in1	100.000	10.000 - 800.000	0.010	Single ended clock capable pin
☐ Secondary	clk_in2	100.000	60.000 - 120.000	0.010	Single ended clock capable pin

OK Cancel

Re-customize IP

Clocking Wizard (6.0)

Documentation IP Location Switch to Defaults

IP Symbol Resource

☒ Show disabled ports

Component Name sys_dcm

Clocking Options Output Clocks Port Renaming MMCM Settings Summary

Output Clock	Port Name	Requested	Actual	Requested	Actual	Requested	Actual	Drives
☒ clk_out1	clk_out1	100.000	100.00000	0.000	0.000	50.000	50.0	BUFG
☒ clk_out2	clk_out2	12.500	12.50000	0.000	0.000	50.000	50.0	BUFG
☐ clk_out3	clk_out3	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out4	clk_out4	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out5	clk_out5	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out6	clk_out6	100.000	N/A	0.000	N/A	50.000	N/A	BUFG
☐ clk_out7	clk_out7	100.000	N/A	0.000	N/A	50.000	N/A	BUFG

☐ USE CLOCK SEQUENCING

Clocking Feedback

Output Clock	Sequence Number	Source	Signaling
clk_out1	1	☒ Automatic Control On-Chip	☒ Single-ended
clk_out2	1	☐ Automatic Control Off-Chip	☐ Differential
clk_out3	1	☐ User-Controlled On-Chip	
clk_out4	1	☐ User-Controlled Off-Chip	
clk_out5	1		
clk_out6	1		
clk_out7	1		

Enable Optional Inputs / Outputs for MMCM/PLL

☒ reset ☐ power_down ☐ input_clk_stopped

Reset Type

☒ Active High ☐ Active Low

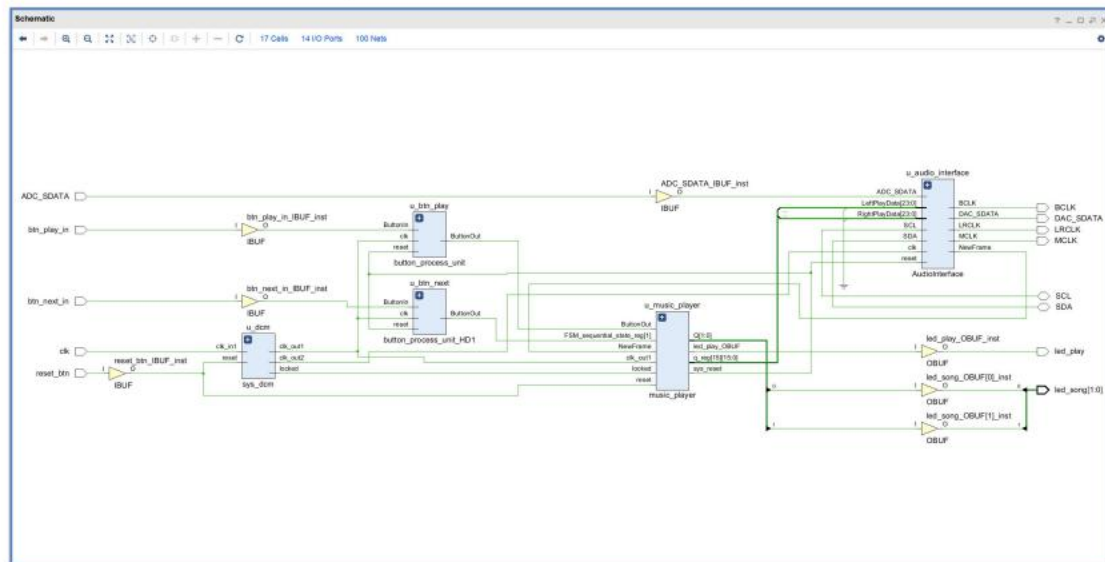
OK Cancel

约束文件应如下：

```
music_player.xdc
E:/AAAzju/Digital_System_EXP/3240105050/lab19/src/MP_xdc/music_player.xdc

1 set_property PACKAGE_PIN U16 [get_ports {led_song[1]}]
2 set_property PACKAGE_PIN T16 [get_ports {led_song[0]}]
3 set_property PACKAGE_PIN T4 [get_ports ADC_SDATA]
4 set_property PACKAGE_PIN T5 [get_ports BCLK]
5 set_property PACKAGE_PIN D22 [get_ports btn_next_in]
6 set_property PACKAGE_PIN D14 [get_ports btn_play_in]
7 set_property PACKAGE_PIN R4 [get_ports clk]
8 set_property PACKAGE_PIN W6 [get_ports DAC_SDATA]
9 set_property PACKAGE_PIN T14 [get_ports led_play]
10 set_property PACKAGE_PIN U5 [get_ports LRCLK]
11 set_property PACKAGE_PIN U6 [get_ports MCLK]
12 set_property PACKAGE_PIN B22 [get_ports reset_btn]
13 set_property PACKAGE_PIN W5 [get_ports SCL]
14 set_property PACKAGE_PIN V5 [get_ports SDA]
15 set_property IOSTANDARD LVCMOS33 [get_ports {led_song[1]}]
16 set_property IOSTANDARD LVCMOS33 [get_ports {led_song[0]}]
17 set_property IOSTANDARD LVCMOS33 [get_ports ADC_SDATA]
18 set_property IOSTANDARD LVCMOS33 [get_ports BCLK]
19 set_property IOSTANDARD LVCMOS33 [get_ports btn_next_in]
20 set_property IOSTANDARD LVCMOS33 [get_ports btn_play_in]
21 set_property IOSTANDARD LVCMOS33 [get_ports clk]
22 set_property IOSTANDARD LVCMOS33 [get_ports DAC_SDATA]
23 set_property IOSTANDARD LVCMOS33 [get_ports led_play]
24 set_property IOSTANDARD LVCMOS33 [get_ports LRCLK]
25 set_property IOSTANDARD LVCMOS33 [get_ports MCLK]
26 set_property IOSTANDARD LVCMOS33 [get_ports reset_btn]
27 set_property IOSTANDARD LVCMOS33 [get_ports SCL]
28 set_property IOSTANDARD LVCMOS33 [get_ports SDA]
29 create_clock -period 10.000 -name clk -waveform {0.000 5.000} [get_ports clk]
```

准备完毕后，可以开始综合并上板验证，上板后发现，正常播放，设计正确。



6 Debug 过程

在将代码进行下板测试时，遇到了诸多工程问题。通过查阅教材与波形分析推理，借助 AI 的辅助，我逐一解决了这些 Bug，以下为 Debug 记录：

6.1. "MDRV-1" 多驱动网络冲突报错

第一次下板时，Vivado 综合报错 [DRC MDRV-1] Multiple Driver Nets，指出 ButtonOut 信

号连到了多个驱动源。

经排查发现，工程中同时存在我添加过 Verilog 代码的 `button_process_unit.v` 以及老师提供的黑盒网表 `button_process_unit.edf`。Vivado 综合时将两者合并，导致两套消抖电路同时去抢占同一个输出管脚。

解决方案是清空 `.v` 文件中的所有逻辑，将其变为纯端口声明，让 Vivado 自动读取底层 `.edf` 网表。

6.2. LED 播放灯一闪即灭

下板后，按下播放键，代表播放状态的 LED 仅仅闪了一下就熄灭，耳机里没有任何音乐，只听到一声急促的爆鸣。

这是代码的问题。为了加速仿真，我曾将节拍分频器的 `DIV_N` 设为 1000。在 100MHz 时钟下，这就产生了每秒 10 万次的节拍脉冲（Beat），导致一首几百拍的歌曲在 10 毫秒内播完，随后状态机接收到 `song_done` 信号回到暂停状态。解决方案是在 `music_player.v` 中引入 `sim` 参数，下板时将分频比改为 2,000,000，产生约 50Hz 的节拍，成功恢复正常播放速度。

6.3. 状态机跳转

播放时，发现按 `next` 后系统依然陷入暂停。

检查代码发现，是 MCU 的状态转换不正确，NEXT 状态后，下一状态应为 PLAY。

6.4. DCM 时钟管理配置缺失

最初设计时，直接将板载的 100MHz 晶振时钟强制供给系统所有模块，导致 LED 灯一闪即灭，耳机里只听到一声急促的爆鸣。同时，在后期引入 `dcm_core` 时发现 DCM 模块未与其他模块连接，应该是次顶层模块的问题。

系统是一个典型的双时钟域架构，音频芯片有严格的硬件采样率时钟需求（必须为 12.288MHz），而主逻辑跑在 100MHz。直接使用 100MHz 会导致系统功能异常。

重新生成 IP 核并连接至整个系统中去后，DCM 正常接入并提供时钟，将外部输入分频为 `sys_clk` 和 `audio_clk`。

7 思考题

7.1. 在实验中，为什么 `next_button`、`play_pause_button` 两个按键需要消抖动及同步化处理，而 `reset` 按键不需要消抖动及同步化处理？

因为 `next` 和 `play_pause` 信号在 MCU 中被用作状态机跳转的核心触发条件。如果不进行消抖，一次物理按键产生的多次高低电平毛刺，会被 100MHz 的高速系统时钟识别为几百上千次按键动作，导致状态机陷入混乱的跳转；如果不进行同步化处理，异步按键信号极易导致状态寄存器产生亚稳态从而使系统崩溃。

而 `reset` 信号的作用是将系统强制初始化。在本项目中，即使机械按键产生毛刺，其结果也只是让系统处于复位状态的时间多延长了几毫秒而已，这段时间内的抖动无论如何都不会引发逻辑错误。因此从节省逻辑资源的角度出发，`reset` 按键不需要进行消抖和同步化处理。

7.2. 在主控制器(mcu)设计中，是否存在接收不到按键信息的可能？若存在，概率多大？有没有必要修改设计？

存在接收不到按键信息的可能性。

按键经过消抖和边沿检测模块后，输出的是一个精确持续 1 个时钟周期（系统时钟 100MHz 下为 10ns）的正脉冲信号。主控状态机（MCU）在 PAUSE 和 PLAY 状态下均能

正常响应此脉冲。然而，在 MCU 处于 RESET 和 NEXT 这两个状态时，其次态逻辑是无条件转移（例如离开 NEXT 状态时无条件 `next_state = PLAY`），在代码中根本没有去检测 `play_pause` 或 `next` 信号。如果消抖后的单周期脉冲恰好、且仅仅发生在这 10ns 的“盲区”周期内，状态机就会彻底漏掉这次按键信息。

但是，它的发生概率极低，几乎可以忽略不计。因为 RESET 和 NEXT 状态在整个运行时间轴上仅持续短短的 1 个时钟周期。而人类的正常反应与连续按键间隔至少在数百毫秒级别。因此，有效按键脉冲恰好撞在这 10ns 盲区上的概率仅为数千万分之一，在实际人为操作中几乎不可能发生。

因此完全没有必要修改设计。工程设计讲究成本与收益的平衡。一方面，该现象发生的概率微乎其微，即便遭遇极小概率漏按，用户只需本能地再按一次即可恢复，绝不会导致系统死机或崩溃；另一方面，如果为了防范这千万分之一的概率去强行增加输入脉冲缓存寄存器，会徒增电路复杂度和功耗。因此，维持当前设计是最具工程性价比的选择。

8 总结与心得

本次 FPGA 音乐播放器大作业是我本科阶段目前为止经历过最完整、也最具挑战性的数字系统设计实践。我完成了整个系统的架构设计、Verilog 代码编写、仿真以及综合测试工作。在这个过程中，我巩固了控制器、计数器、同步电路等理论知识。

一开始的子实验 DDS 设计充满了挑战。正弦表对我来说是一个全新的概念，加上数模转换的理论知识尚未深入，给理解设计思路造成了不小的阻碍。此外，地址处理、数据处理这两个核心模块需要基于实验原理独立设计，我花费了较长时间去弄懂。

跨过底层难关后，正式着手音乐播放器的整体其实也不简单。因为缺乏状态机代码的训练，在写代码的过程中花费了不少时间，即使有了教材的原理框图和前面一系列 Lab 的训练，根据 ASM 图设计各模块的控制器也不那么简单。我仿照教材的示例代码，慢慢完成了第一个 mcu 模块的控制器设计，并在此过程中掌握了 FSM 的 Verilog 编写流程。几个子模块的控制器设计让我深刻体会到了“自顶向下（Top-Down）”与模块化设计的优势。

毕竟是第一次设计如此庞大的工程，实验中我还是遇到了大大小小的问题。这些问题已经在 6 Debug 过程中有过叙述，因此不再赘述。

前前后后耗时八周，最初对着 DDS 相位累加器一头雾水，到后来也能够完整地写出三段式状态机代码，Verilog 能力有了一定的进步，也体会到了“硬件编程”与“软件编程”的区别。Verilog 不是在“写程序”，而是在“搭电路”。每一个时钟周期的延迟、每一根连线的位宽匹配、每一个信号的默认状态，都会在物理世界中反映。此外，关于 FSM 组合逻辑中避免 Latch 的编写规范，更是让我受益匪浅。

最终音频正常播放的那一刻，我的内心是十分激动的。通过本学期的实验，我不仅掌握了 Verilog 语言和数字系统设计的基本流程，更将零散的知识点串联成了一个片上系统。做完实验后，我深感实际的电路设计充满挑战。毕竟我在有教材、有老师辅导、有设计流程提示，甚至包括 Vibe Coding 的情况下尚且感到困难重重，完全独立的工业级数字系统设计显然具有远高于此的能力要求。感谢两位老师的悉心指导。这次实验让我意识到，在这个领域我还需要学习大量知识、进行大量练习来提升自己，而这次积累的宝贵经验，将是我未来面对更复杂硬件工程问题时最坚实的底气。

最后我想说的是个人对于 Vibe Coding 的一点看法。不得不承认，在完成整个实验的过程中 AI 确实帮了我很多的忙，前面说到，我是缺乏状态机代码的训练的，在完成本项目之前，我甚至不知道什么是一段式、二段式与三段式状态机。不过，在完成项目的时候，我一点一点地询问 Gemini、向它学习了三段式状态机的写法。虽然学在浙大的视频里教的是两段式，但是 Gemini 教给我的三段式对我来说更容易理解，因此我的所有代码、包括考试的

时候写出来的基本都是三段式结构。有时候遇到一些不理解的问题我也会先试着问问 Gemini 尝试自己解决，实在解决不了的才会问老师，这样也能提高我完成项目的速度，虽然其实验收的速度也不如很多同学，毕竟我的代码基础确实不算很好，总会遇到许许多多的小问题甚至大问题。

但是，我并不支持 Vibe Coding 的大量使用。就像老师们说到的，AI 的代码确实能帮你很快的完成项目，可你自己却没有对代码的一点理解，一方面，它会让你越来越依赖 Vibe Coding，另一方面，AI 也仅仅是在这类较小的项目上较为得心应手，当代码量变大、涉及到多个项目之间的串联的时候，AI 往往会写出仅在某一个项目内正确的代码，但其他项目与这个项目连接后就会出现大问题。所以，我个人认为，Vibe Coding 仅仅是能部分提升效率的手段，而绝不能完全的依赖它。